

Mục lục

Lời cảm ơn	3
Tóm tắt	4
Chương 1. Giới thiệu	5
1.1 Lý do Chọn Đề tài.....	5
1.2 Động lực.....	5
1.3 Luồng Thiết kế Tổng quan	6
Chương 2: Cơ sở Lý thuyết.....	7
2.1 Kiến trúc Accelerator AI và Vai trò của Mảng MAC	7
2.2 Compute-in-Memory (CIM) và Vai trò của Khối “Fallback” Số	7
2.3 Dataflow trong Accelerator: Weight-Stationary và Các Kiểu Khác	8
2.4 Lượng tử hóa INT8/INT4 trong Suy luận Mạng Neural	9
2.5 Luồng Thiết kế RTL-to-GDSII.....	9
2.5.1 Logic Synthesis	9
2.5.2 Place-and-Route (P&R)	10
2.5.3 Static Timing Analysis và Sign-off.....	10
Chương 3. Tổng quan Kết quả.....	11
3.1 Kết quả Kiểm tra Chức năng	11
2.2 Kết quả Triển khai Vật lý (OpenLane)	11
2.3 Bảng Tổng hợp	12
Chương 4: Kiến trúc và Thiết kế Chi tiết.....	13
4.1 Kiến trúc Tổng quan	13
4.2 Processing Element (PE) — pe.v.....	14
4.3 Mảng MAC — mac.v	14
4.4 Bộ Điều khiển Top-Level — top.v	15
4.5 Tham số Thiết kế	16
Chương 5: Kiểm tra Chức năng	17
5.1 Môi trường Mô phỏng	17
5.2 Quy trình Chạy Mô phỏng.....	17
5.3 Kiến trúc Testbench (tb_top1.sv)	18

5.4 Test Case và Kết quả Chi tiết	18
Chương 6: Open Source (OpenLane)	20
6.1 Tổng quan Luồng.....	20
6.2 Phân tích Vấn đề Routing ($N = 2$ so với $N = 8$)	21
6.3 Kết quả Chi tiết theo Từng Giai đoạn.....	21
<i>Synthesis</i>	21
<i>Placement và Routing</i>	22
<i>Static Timing Analysis (STA)</i>	23
<i>Power</i>	24
6.4 Tổng kết PPA.....	25
Chương 7: Kết luận và Hướng phát triển.....	26
7.1 Kết luận.....	26
7.2 Hạn chế Tổng thể.....	26
7.3 Hướng Phát triển.....	27
7.4 Mã nguồn	28
Tài liệu Tham khảo	29

Mục lục hình ảnh

Hình 1: kết quả testbench.....	11
Hình 2 . kết quả full flow success	11
Hình 3. Sơ đồ khối mức hệ thống của module top-level.	13
Hình 4. Datapath của processing element (PE).	14
Hình 5. Dataflow của mảng MAC_ARRAY từ khối PE.....	15
Hình 6. FSM điều khiển ở top-level (top.v).....	16
Hình 7. Dạng sóng thực tế	19
Hình 8. Các giai đoạn luồng RTL-to-GDSII của OpenLane.	20
Hình 9. Kết quả Synthesis bằng Yosys.....	22
Hình 10. Layout GDSII của MAC INT8 sau khi hoàn tất OpenLane fullflow.	23
Hình 11. kiểm tra DRC/LVS bằng các công cụ trong flow OpenLane	25

Lời cảm ơn

Trong suốt quá trình thực hiện đề tài, em đã nhận được sự quan tâm, hướng dẫn và góp ý tận tình của Thầy Hoàng Mạnh Hà. Những định hướng của Thầy về kiến trúc thiết kế số, luồng triển khai RTL-to-GDSII và cách tiếp cận bài toán NPU đã giúp em hiểu rõ hơn quy trình thiết kế vi mạch số từ mức RTL đến triển khai vật lý.

Em xin chân thành cảm ơn Bộ môn Điện tử – Khoa Điện - Điện tử, Trường Đại học Bách Khoa TP.HCM đã tạo điều kiện về môi trường học tập, tài liệu kỹ thuật và định hướng nghiên cứu để em có thể thực hiện đề tài. Thông qua quá trình thiết kế, mô phỏng, tổng hợp và thử nghiệm với OpenLane, em đã có thêm kinh nghiệm thực tế trong việc xây dựng một khối tính toán số phục vụ cho kiến trúc NPU.

Mặc dù đã cố gắng trong quá trình thực hiện, báo cáo vẫn khó tránh khỏi những thiếu sót do kiến thức và kinh nghiệm còn hạn chế, đặc biệt trong các bước triển khai vật lý và đánh giá sau tổng hợp. Em rất mong nhận được sự nhận xét và góp ý từ Thầy để có thể hoàn thiện hơn trong các nghiên cứu và đồ án tiếp theo.

Tóm tắt

Đề tài nghiên cứu và hiện thực một khối tính toán số cho NPU dạng MAC Array INT8 weight-stationary, phục vụ vai trò datapath số “fallback” trong các kiến trúc NPU. Thiết kế được mô tả bằng ngôn ngữ Verilog HDL, gồm ba module chính: top.v, mac.v và pe.v. Trong đó, top.v đóng vai trò module điều khiển mức cao, sử dụng FSM ba trạng thái IDLE/RUN/DONE để lần lượt chọn từng hàng của ma trận đầu vào A, mac.v thực hiện phép nhân ma trận theo dạng dot-product giữa hàng A đang xét và ma trận trọng số B, và pe.v là phần tử tính toán cơ bản, thực hiện phép nhân có dấu INT8 và cộng dồn vào kết quả 32-bit.

Thiết kế được xây dựng theo hướng tham số hóa, trong đó cấu hình mục tiêu ban đầu là mảng MAC kích thước $N = 8$. Tuy nhiên, để hoàn tất đầy đủ luồng triển khai vật lý trên môi trường máy cá nhân, cấu hình $N = 2$ được sử dụng làm bản demo full-flow. Với cấu hình này, mảng MAC gồm $2 \times 2 = 4$ PE, vẫn giữ nguyên nguyên lý hoạt động của kiến trúc weight-stationary nhưng giảm đáng kể số lượng IO và độ phức tạp routing.

Kết quả kiểm tra chức năng bằng testbench cho thấy thiết kế thực hiện đúng các phép toán nhân rồi cộng có dấu với nhiều trường hợp dữ liệu khác nhau, bao gồm số dương, số âm, số 0 và hỗn hợp dấu. Sau đó, thiết kế được đưa qua luồng RTL to GDSII bằng OpenLane trên công nghệ SkyWater SKY130A với thư viện standard cell sky130_fd_sc_hd. Kết quả cho thấy cấu hình $N = 2$ hoàn tất toàn bộ luồng từ RTL, synthesis, floorplanning, placement, routing đến tạo file GDSII, chứng minh tính khả thi của kiến trúc MAC Array INT8 ở mức triển khai vật lý. Cấu hình $N = 8$ được ghi nhận là hướng mở rộng tiếp theo, cần tối ưu thêm về pipeline, giao tiếp dữ liệu và tài nguyên routing để có thể hoàn tất full-flow ở quy mô lớn hơn.

Chương 1. Giới thiệu

1.1 Lý do Chọn Đề tài

Đề tài được lựa chọn vì phù hợp với định hướng chuyên ngành Thiết kế IC, đồng thời cho phép thực hành trọn vẹn một luồng thiết kế ASIC hoàn chỉnh từ RTL, kiểm tra chức năng, đến synthesis và place and route thay vì chỉ dừng ở mức mô phỏng logic. Đây cũng là một khối chức năng thực sự cần thiết cho các dự án NPU/CIM SoC nói chung: một lõi compute-in-memory (CIM) mang lại hiệu năng năng lượng tốt nhưng có rủi ro công nghệ cao ở lần tape-out đầu, nên một mảng MAC số dùng chung dataflow làm phương án dự phòng là cần thiết để đảm bảo SoC vẫn hoạt động được ngay cả khi lõi CIM chưa đạt yêu cầu. Thực hiện đề tài này là cơ hội tiếp cận đồng thời hai mảng kiến thức quan trọng của thiết kế IC số: kiến trúc accelerator cho AI (dataflow, hệ thống MAC array) và quy trình vật lý đưa thiết kế từ RTL đến GDSII bằng công cụ opensource.

1.2 Động lực

Các lõi NPU compute-in-memory (CIM) hứa hẹn cải thiện hiệu năng năng lượng lớn so với datapath số thông thường. Một khối (a) được map bởi cùng compiler và dataflow dùng để lập lịch cho mảng CIM, (b) chứng minh được qua một luồng synthesis và place-and-route ASIC thông thường thay vì luồng analog tùy biến, và (c) có thể thay thế an toàn nếu lõi CIM không đạt yêu cầu. Các workload mục tiêu keyword spotting hoạt động liên tục và tầm soát bất thường/loạn nhịp ECG đều là mô hình nhỏ, phù hợp với INT8, chạy trên thiết bị biên/đeo được với ràng buộc năng lượng chặt chẽ đó là lý do một mảng MAC INT8 được chọn làm khối tính toán lõi thay vì một datapath dấu phẩy động đầy đủ.

1.3 Luồng Thiết kế Tổng quan

Dự án đi theo hai hướng triển khai song song.

- Hướng thứ nhất là luồng ASIC: thiết kế RTL → mô phỏng tự kiểm tra (self-checking simulation) → synthesis, static timing analysis, và place-and-route qua luồng RTL-to-GDSII mã nguồn mở OpenLane [\[4\]](#) → sign-off DRC / LVS → GDSII. (Hoàn thành)
- Hướng thứ hai là dựng thử trên FPGA, board Altera DE2 qua Quartus II, dùng như một cách kiểm tra chức năng độc lập cho cùng RTL. (Đang tiến hành)

Chương 2: Cơ sở Lý thuyết

2.1 Kiến trúc Accelerator AI và Vai trò của Mảng MAC

Phần lớn khối lượng tính toán trong suy luận (inference) mạng neural quy về phép nhân ma trận: đầu ra của một lớp mạng là tổng có trọng số của các đầu vào, tức là tích vô hướng (dot product) giữa vector activation và vector trọng số, lặp lại cho hàng ngàn đến hàng triệu cặp giá trị. Một CPU đa dụng thực hiện các phép nhân–cộng này tuần tự hoặc bán song song, dẫn đến hiệu năng và hiệu suất năng lượng thấp so với yêu cầu của thiết bị biên.

Một mảng MAC (multiply-accumulate array) giải quyết vấn đề này bằng cách bố trí nhiều processing element (PE) — mỗi PE chỉ làm một phép nhân và một phép cộng dồn — thành một lưới tính toán song song. Với cách tổ chức phù hợp (dataflow), một mảng $N \times N$ PE có thể tính đồng thời N tích vô hướng mỗi chu kỳ, khai thác song song hóa phần cứng thay vì tốc độ xung nhịp để đạt hiệu năng cao với công suất thấp — phù hợp cho các workload biên như keyword spotting và ECG.

2.2 Compute-in-Memory (CIM) và Vai trò của Khối “Fallback” Số

Trong kiến trúc số truyền thống (von Neumann), dữ liệu (trọng số, activation) phải được đọc từ bộ nhớ, di chuyển đến đơn vị tính toán, rồi kết quả được ghi lại — việc di chuyển dữ liệu này thường tiêu tốn năng lượng nhiều hơn chính phép tính. Compute-in-memory (CIM) giảm chi phí này bằng cách thực hiện phép MAC ngay bên trong mảng bộ nhớ: trọng số được lưu tại chỗ trong các bit-cell, và phép nhân–cộng dồn được thực hiện thông qua đặc tính vật lý của mảng (ví dụ cộng dòng điện trên bit-line), tránh phải đọc từng trọng số ra ngoài.

Tuy tiết kiệm năng lượng, CIM còn là công nghệ đang phát triển: mạch đọc analog, biến thiên linh kiện, và nhiễu lượng tử hóa ADC có thể ảnh hưởng đến độ chính xác hoặc yield ở lần chế tạo đầu tiên. Vì vậy, nhiều dự án SoC tích hợp CIM đi kèm một khối “dự phòng” (fallback) hoàn toàn số, dùng chung sơ đồ ánh xạ dữ liệu (dataflow) và compiler với lõi CIM. Nếu lõi CIM không đạt yêu cầu, khối fallback vẫn đảm bảo hệ thống hoạt động được, đồng thời khối này có thể chứng minh được qua một luồng ASIC tiêu chuẩn (không cần luồng analog tùy biến).

2.3 Dataflow trong Accelerator: Weight-Stationary và Các Kiểu Khác

Dataflow của một accelerator mô tả cách dữ liệu (trọng số, activation, tổng riêng phần) di chuyển qua mảng PE, và toán hạng nào được giữ cố định (stationary) trong mỗi PE để giảm số lần truy xuất bộ nhớ. Ba kiểu phổ biến [2]:

- Weight-stationary: trọng số được nạp vào từng PE một lần và giữ cố định; activation được stream qua mảng. Giảm số lần đọc trọng số — phù hợp khi trọng số được tái sử dụng nhiều lần (như trong lớp fully-connected hoặc convolution).
- Output-stationary: tổng riêng phần (partial sum) được giữ cố định tại mỗi PE trong khi trọng số và activation được stream qua; giảm số lần đọc/ghi thành ghi tích lũy.
- Row-stationary (hybrid, ví dụ Eyeriss): kết hợp cả ba kiểu trên theo từng chiều của phép tính để tối ưu tổng thể năng lượng di chuyển dữ liệu, đánh đổi độ phức tạp điều khiển cao hơn.

Thiết kế trong đề tài này chọn weight-stationary cụ thể vì nó mô phỏng đúng cách một mảng bit-cell CIM hoạt động: trọng số được ghi vật lý vào mảng một lần, còn

activation được stream vào và nhân trực tiếp tại chỗ (Mục 2.2). Nhờ vậy, code compiler/tiling dùng để ánh xạ một mạng neural lên lõi CIM cần ít hoặc không cần thay đổi để cũng nhắm tới khối fallback này. NVDLA [3], accelerator suy luận Verilog mã nguồn mở của NVIDIA, và kiến trúc systolic dùng trong TPU của Google [1] là các điểm tham chiếu cho một lõi accelerator số, mã nguồn mở, xoay quanh mảng MAC ở cùng tầng kiến trúc.

2.4 Lượng tử hóa INT8/INT4 trong Suy luận Mạng Neural

Mạng neural thường được huấn luyện ở định dạng dấu phẩy động (FP32), nhưng với nhiều tác vụ suy luận đặc biệt các mô hình nhỏ như keyword spotting hay phân loại tín hiệu ECG trọng số và activation có thể được lượng tử hóa (quantize) về số nguyên 8-bit (INT8) hoặc thậm chí 4-bit (INT4) mà độ chính xác suy giảm không đáng kể. Lượng tử hóa mang lại ba lợi ích chính cho phần cứng: (1) giảm dung lượng bộ nhớ lưu trọng số, (2) đơn vị số học nhân–cộng số nguyên đơn giản, nhanh, và tiêu thụ năng lượng thấp hơn nhiều so với đơn vị dấu phẩy động tương đương, và (3) diện tích mạch nhỏ hơn cho cùng một mức song song hóa.

Vì các lý do đó, thiết kế trong đề tài dùng INT8 làm định dạng số học mặc định cho mảng MAC, với INT4 được quy hoạch như một chế độ mở rộng (Mục 7.3) để giảm thêm diện tích/năng lượng cho các mô hình chấp nhận được độ chính xác thấp hơn.

2.5 Luồng Thiết kế RTL-to-GDSII

Để đưa một thiết kế RTL thành GDSII file layout vật lý cuối cùng dùng để chế tạo thiết kế phải đi qua ba nhóm bước chính, tóm tắt trong các mục sau và minh họa cụ thể bằng công cụ OpenLane ở Chương 6.

2.5.1 Logic Synthesis

Synthesis chuyển RTL (mô tả hành vi bằng Verilog) thành netlist mức cổng logic (gate-level), sử dụng các cell chuẩn (standard cell) từ thư viện của công nghệ mục tiêu (PDK). Công cụ synthesis tối ưu netlist theo các ràng buộc về diện tích, tốc độ, và công suất do người thiết kế cung cấp (thường ở định dạng SDC). Trong luồng OpenLane, bước này do Yosys đảm nhiệm.

2.5.2 Place-and-Route (P&R)

Place-and-Route gồm nhiều bước con: floorplanning xác định kích thước và hình dạng vùng die/core cùng vị trí các chân I/O; placement gán vị trí vật lý cụ thể cho từng cell chuẩn trong netlist; clock tree synthesis (CTS) xây dựng mạng phân phối xung nhịp cân bằng đến mọi thanh ghi; và routing tạo các đường dây kim loại (metal) thực sự nối các chân cell theo đúng netlist. Trong OpenLane, các bước này do OpenROAD thực hiện.

2.5.3 Static Timing Analysis và Sign-off

Static timing analysis (STA) kiểm tra rằng mọi tín hiệu đến đúng thanh ghi đích trong thời gian một chu kỳ xung nhịp, có tính đến trễ của cell và dây dẫn thực tế sau layout, báo cáo *slack* (dư thời gian) cho từng đường tín hiệu; nếu slack âm ở bất kỳ đường nào, thiết kế chưa đạt *timing closure* và cần synthesis/P&R lại hoặc sửa RTL. Sau khi đạt timing, thiết kế còn phải qua hai bước sign-off vật lý: DRC (design rule check) kiểm tra layout tuân thủ luật hình học của quy trình chế tạo, và LVS (layout versus schematic) đối chiếu layout với netlist gốc để đảm bảo không có sai khác. Chỉ khi đạt cả STA, DRC, và LVS, thiết kế mới sẵn sàng xuất ra GDSII.

Chương 3. Tổng quan Kết quả

3.1 Kết quả Kiểm tra Chức năng

RTL của cả ba module (*pe.v*, *mac.v*, *top.v*) đã hoàn chỉnh và được kiểm tra bằng testbench tự kiểm tra chạy trong Verilator Verilog.

Kết quả: 5/5 test case PASS, bao phủ các trường hợp toán hạng dương, âm, hỗn hợp dấu, và bằng 0. Phương pháp và chi tiết từng test case :

```
● nhandeptrai@DESKTOP-K1RQKRI:~/npu$ ./obj_dir/Vtb_top1 | tee tb_top1_verilator_result.txt
CASE 1 PASSED: A=1, B=1, C=8
busy = 0, done = 1
CASE 2 PASSED: A=0, B=1, C=0
busy = 0, done = 1
CASE 3 PASSED: A=-1, B=1, C=-8
busy = 0, done = 1
CASE 4 PASSED: A=2, B=-3, C=-48
busy = 0, done = 1
CASE 5 PASSED: A=-2, B=-2, C=32
busy = 0, done = 1
ALL TEST CASES PASSED
- tb_top1.sv:154: Verilog $finish
```

Hình 1: kết quả testbench

2.2 Kết quả Triển khai Vật lý (OpenLane)

Thiết kế đang được đưa từ RTL sang GDSII bằng luồng mã nguồn mở OpenLane. Kết quả tính đến thời điểm báo cáo: ở cấu hình mặc định **N = 8** (64 PE), giai đoạn detailed routing không thể hoàn thiện trên máy hiện có vì thiết kế đòi hỏi số lớp routing nhiều hơn mức máy có thể xử lý xong. Ở cấu hình rút gọn **N = 2** (4 PE), có thể đi full flow từ đầu đến cuối, xác nhận toàn bộ luồng RTL-to-GDSII chạy được.

```
● nhandeptrai@DESKTOP-K1RQKRI:~/OpenLane/designs/npu_macarray/runs/N2_success$ grep -i "flow complete\\|success" openlane.log
[SUCCESS]: Flow complete.
```

Hình 2 . kết quả full flow success

2.3 Bảng Tổng hợp

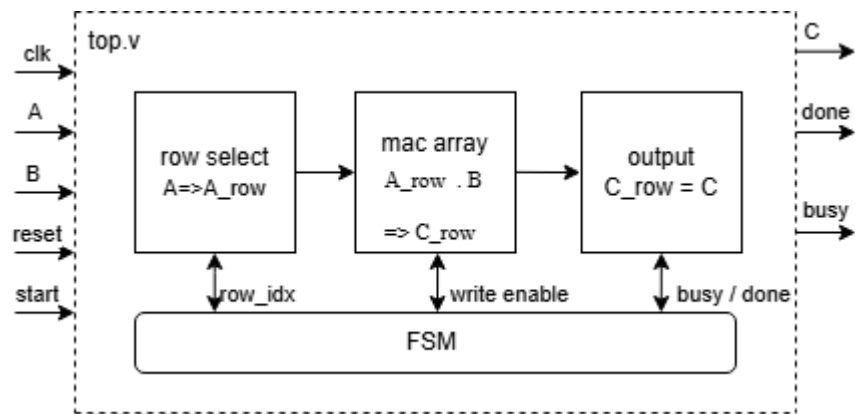
Hạng mục	Kết quả
RTL & kiến trúc	Hoàn chỉnh: PE, mảng MAC $N \times N$, FSM top-level; tham số hóa $N = 8$, INT8, accumulator 32-bit
Kiểm tra chức năng (simulation)	5/5 test case PASS (Icarus Verilog), 0 sai lệch / 320 word kiểm tra
Triển khai vật lý — $N = 2$	Route thành công, full flow chạy xong — [điền số liệu PPA cụ thể]
Triển khai vật lý — $N = 8$ (mục tiêu)	Chưa route xong — vượt số lớp routing khả dụng trên máy hiện có
Demo FPGA (DE2 / Quartus II)	Chưa thực hiện

Bảng 1. Tổng hợp kết quả tính đến thời điểm báo cáo.

Chương 4: Kiến trúc và Thiết kế Chi tiết

4.1 Kiến trúc Tổng quan

Thiết kế gồm ba module *pe.v*, *mac.v*, và *top.v* thể hiện ở mức hệ thống trong Hình 3. Một bộ điều khiển bên ngoài phát xung *start*; FSM ở top-level sau đó stream một hàng của ma trận activation A mỗi chu kỳ xung nhịp vào mảng MAC, nhân với ma trận trọng số cố định B, và chụp lại một hàng của ma trận kết quả C mỗi chu kỳ cho đến khi tất cả N hàng đã được xử lý.



Hình 3. Sơ đồ khối mức mức hệ thống của module top-level.

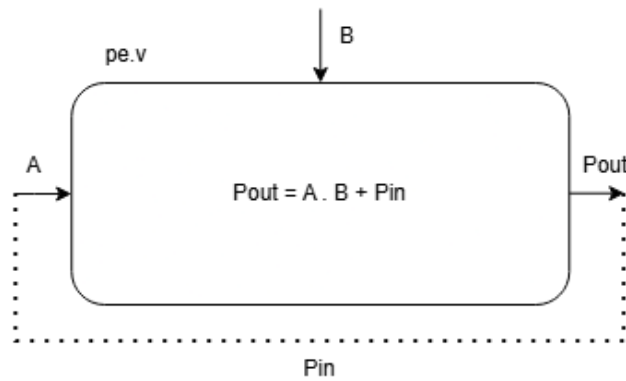
Bảng 2 tóm tắt các module chính trong thiết kế và testbench.

Module	Chức năng	Ghi chú
pe	Processing Element (PE)	Nhân 8-bit có dấu, cộng dồn 32-bit, tổ hợp thuần túy
mac	Mảng MAC N×N	Ghép N×N PE kiểu weight-stationary, tính $C = A_row \cdot B$
top	Bộ điều khiển top-level	FSM IDLE/RUN/DONE, stream từng hàng A qua mac, chụp kết quả C
tb_top1	Testbench Icarus Verilog	5 test case tự kiểm tra, toán hạng dương/âm/hỗ hợp dấu/bằng 0

Bảng 2. Danh sách module chính trong thiết kế.

4.2 Processing Element (PE) — **pe.v**

Mỗi processing element thực hiện một phép nhân có dấu 8×8-bit và cộng dồn kết quả vào một tổng chạy (running sum) 32-bit, thể hiện ở Hình 4. Tích của phép nhân được sign-extend từ 16 lên $ACC_WIDTH = 32$ bit trước khi cộng với tổng riêng phần pin nhận từ PE trước đó trong cùng cột, tạo ra $pout$:

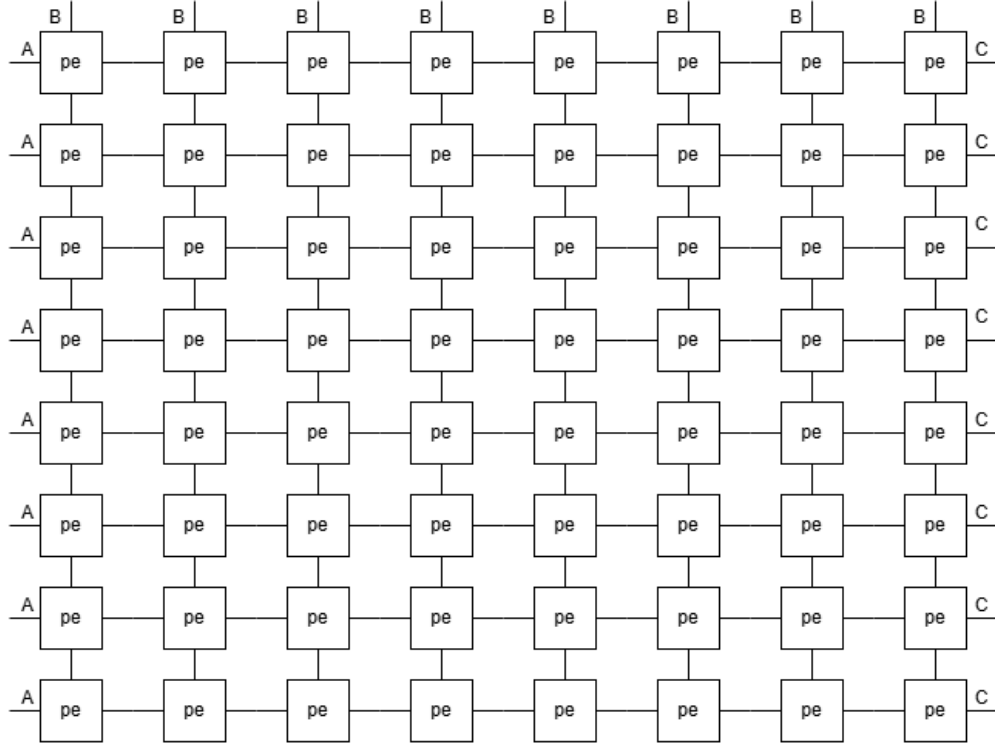


Hình 4. Datapath của processing element (PE).

PE hoàn toàn tổ hợp (combinational) — không có thanh ghi pipeline bên trong. $Pout$ là hàm trực tiếp của A , B , và Pin trong cùng một chu kỳ xung nhịp. Điều này giữ RTL đơn giản cho giai đoạn bring-up chức năng, nhưng có nghĩa là đường critical path xuyên qua mảng sẽ dài ra theo N .

4.3 Mảng MAC — **mac.v**

Mảng MAC (Hình 5) khởi tạo một lưới $N \times N$ các PE bằng khối *generate*. Trọng số B được giữ cố định (stationary) trên toàn mảng trong suốt quá trình tính một hàng activation; hàng activation A được broadcast theo chiều ngang qua từng hàng PE, và các tổng riêng phần cộng dồn theo chiều dọc xuống từng cột, sao cho cột j của mảng tính ra một phần tử đầu ra:

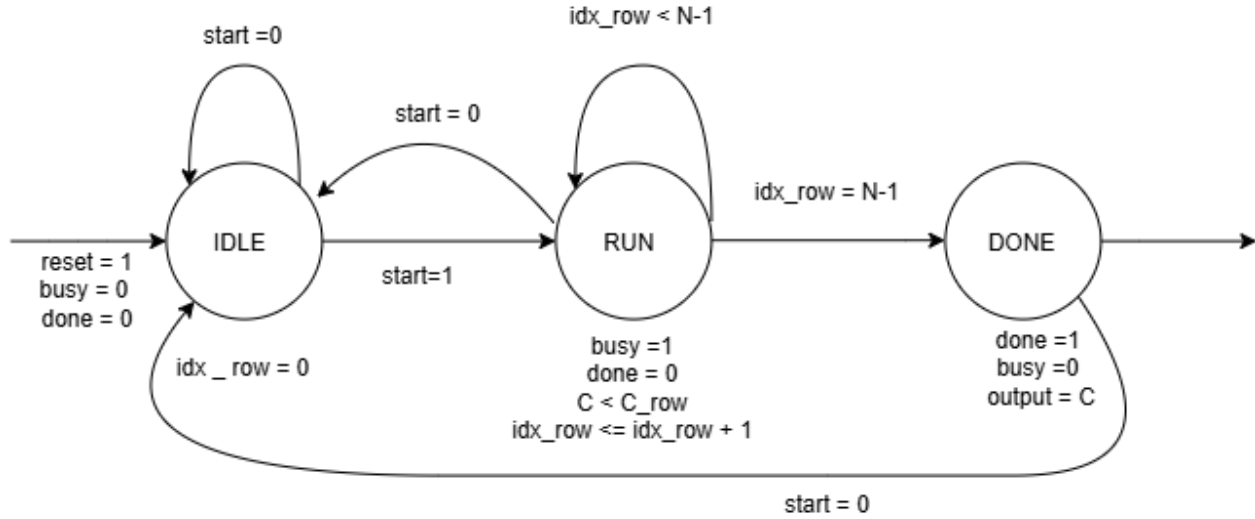


Hình 5. Dataflow của mảng MAC_ARRAY từ khối PE

Tức là mảng tính tích của một vector hàng $1 \times N$ với ma trận cố định $N \times N$, $C = A_{\text{row}} \times B$, mỗi chu kỳ một lần. Về mặt cấu trúc, mỗi cột PE tái tạo đúng kiểu truy cập của một cột bit-cell trong CIM trọng số giữ cố định tĩnh trong từng cell, đường activation broadcast, và cộng dồn theo chiều dọc đây chính là đặc tính cụ thể giữ cho mảng số này tương thích dataflow với compiler CIM.

4.4 Bộ Điều khiển Top-Level — top.v

Module *top* bọc một instance của mảng MAC trong một FSM 3 trạng thái (Hình 4). Khi *start* được kích hoạt, FSM chuyển từ *IDLE* sang *RUN* và, mỗi chu kỳ, chọn hàng *row_idx* của *A*, đưa vào mảng cùng với toàn bộ ma trận cố định *B*, và chốt (latch) hàng kết quả *C* tương ứng vào thanh ghi đầu ra. Sau $N = 8$ hàng, FSM kích hoạt *done* và chuyển sang *DONE*, giữ nguyên ma trận *C* đã hoàn thành cho đến khi *start* được hạ xuống.



Hình 6. FSM điều khiển ở top-level (top.v).

4.5 Tham số Thiết kế

Bảng 3 liệt kê các tham số RTL dùng cho bản build kiểm tra chức năng ở Chương 5; cùng các giá trị này là điểm khởi đầu cho lần chạy OpenLane ở Chương 6.

Tham số	Giá trị	Mô tả
DATA_WIDTH	8	Độ rộng toán hạng activation / weight (INT8, có dấu)
ACC_WIDTH	32	Độ rộng accumulator / đầu ra mỗi phần tử
N	8	Kích thước mảng ($N \times N$ PE, $N \times N$ toán hạng mỗi ma trận)
IDX_WIDTH	3	Độ rộng bộ đếm row_idx ($\log_2 N$ với $N = 8$)

Bảng 3. Tham số thiết kế RTL.

Chương 5: Kiểm tra Chức năng

5.1 Môi trường Mô phỏng

Công cụ	Thông tin sử dụng
Ngôn ngữ thiết kế	Verilog
Ngôn ngữ testbench	SystemVerilog
Công cụ mô phỏng	Icarus Verilog (iverilog / vvp)
Top mô phỏng	tb_top1.sv
Top RTL	top.v (module top)
Waveform	VCD (top1.vcd), xem bằng GTKWave
Dữ liệu kiểm thử	Toán hạng vô hướng: (+1,+1), (0,+1), (-1,+1), (+2,-3), (-2,-2)

5.2 Quy trình Chạy Mô phỏng

Thiết kế được biên dịch và mô phỏng trực tiếp bằng Verilator Verilog, không cần Makefile riêng cho quy mô hiện tại:

```
verilator -Wall -Wno-UNUSED SIGNAL --binary --timing --trace tb_top1.sv pe.v mac.v top.v --
top-module tb_top1
./obj_dir/Vtb_top1
```

Sau khi mô phỏng, file top1.vcd được sinh ra để quan sát dạng sóng, có thể mở bằng:

```
gtkwave top1.vcd
```

5.3 Kiến trúc Testbench (tb_top1.sv)

Testbench khởi tạo *top* và điều khiển nó qua task *run_uniform_case* tự kiểm tra: áp reset, nạp A và B với một giá trị 8-bit có dấu duy nhất broadcast trên mọi phần tử ($A = \{N \times N\{a_val\}\}$, tương tự cho B), phát xung *start* trong một chu kỳ, chờ *done*, sau đó kiểm tra từng phần tử trong $N \times N = 64$ word của C so với kết quả vô hướng kỳ vọng, đếm số sai lệch theo từng case và tổng số. Vì mọi phần tử của A và mọi phần tử của B đều cùng một giá trị trong mỗi case, kết quả kỳ vọng cho mọi phần tử đầu ra rút gọn còn $N \times (a_val \times b_val)$, giúp test tự kiểm tra được mà không cần một reference model riêng.

5.4 Test Case và Kết quả Chi tiết

Bộ test bao phủ toán hạng dương, bằng 0, âm, và hỗn hợp dấu để kiểm tra logic sign-extension và cộng dồn.z

Case	A	B	C kỳ vọng	C mô phỏng	Kết quả
1	+1	+1	8	8	PASS
2	0	+1	0	0	PASS
3	-1	+1	-8	-8	PASS
4	+2	-3	-48	-48	PASS
5	-2	-2	32	32	PASS

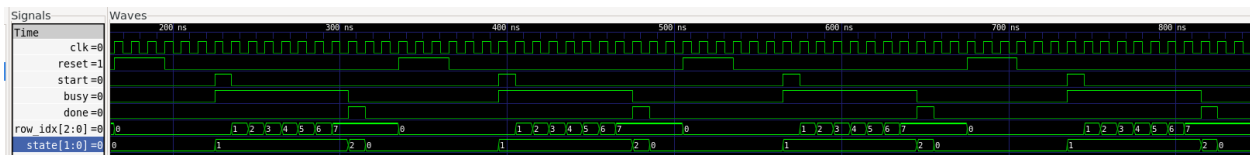
Bảng 4. Kết quả testbench (5/5 case PASS, 0 sai lệch trong 320 word đầu ra được kiểm tra).

Toàn bộ kết quả dưới đây thu được bằng cách compile và chạy thật RTL và testbench trong Icarus Verilog theo đúng quy trình ở Mục 5.2 — đây là log terminal thật, không chỉnh sửa:

```

VCD info: dumpfile top1.vcd opened for output.
CASE 1 PASSED: A=1, B=1, C=8
busy = 0, done = 1
CASE 2 PASSED: A=0, B=1, C=0
busy = 0, done = 1
CASE 3 PASSED: A=-1, B=1, C=-8
busy = 0, done = 1
CASE 4 PASSED: A=2, B=-3, C=-48
busy = 0, done = 1
CASE 5 PASSED: A=-2, B=-2, C=32
busy = 0, done = 1
ALL TEST CASES PASSED
tb_top1.sv:154: $finish called at 845000 (1ps)

```



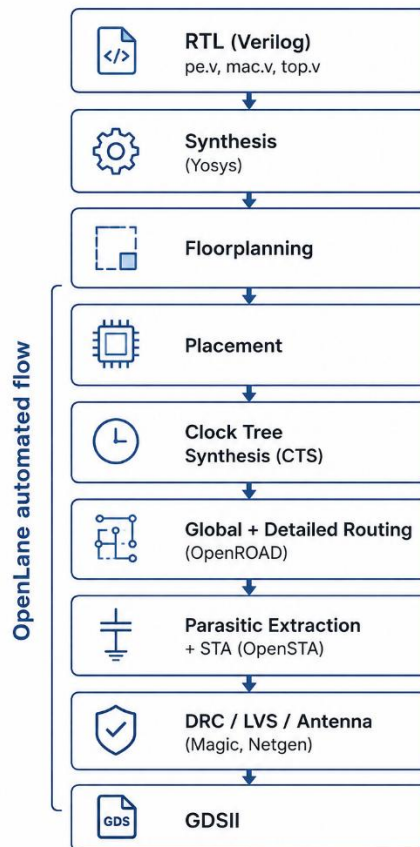
Hình 7. Dạng sóng thực tế thể hiện *start*/*busy*/*done*/*row_idx*/*state* qua cả 5 test case.

Dạng sóng ở Hình 7 khớp với log terminal: mỗi lần *start* lên mức 1, FSM chuyển sang *RUN*, *row_idx* đếm từ 0 đến 7 trong khi *busy* giữ mức 1, sau đó *done* lên mức 1 trong một chu kỳ khi FSM chuyển sang *DONE* đúng 5 lần liên tiếp cho 5 test case, xác nhận cả kết quả số học lẫn timing của handshake *busy* / *done* đều đúng như mô tả ở Mục 4.4.

Chương 6: Open Source (OpenLane)

6.1 Tổng quan Luồng

Triển khai vật lý dùng OpenLane [4], một luồng RTL-to-GDSII tự động mã nguồn mở xây dựng trên Yosys (logic synthesis), OpenROAD (floorplanning, placement, clock-tree synthesis, và routing), OpenSTA (static timing analysis), và Magic / Netgen (DRC, LVS, và antenna check), thể hiện ở Hình 5 — cụ thể hóa các khái niệm đã trình bày ở Mục 2.5. Đây là đối trọng mã nguồn mở của một luồng synthesis + STA + P&R thương mại (vd. Cadence Genus / Innovus), và tạo ra GDSII có thể merge được với foundry ở cuối luồng.



Status: currently running = update this figure/Section ...
with the reached stage and results once the OpenLane run completes.

Hình 8. Các giai đoạn luồng RTL-to-GDSII của OpenLane.

PDK mã nguồn mở mặc định và được hỗ trợ rộng rãi nhất của OpenLane là node SkyWater SKY130nm.

6.2 Phân tích Vấn đề Routing ($N = 2$ so với $N = 8$)

Như đã nêu ở Mục 3.2, ở cấu hình mặc định $N = 8$ (64 PE), giai đoạn detailed routing của OpenLane không hoàn tất trên máy hiện có: thiết kế đòi hỏi số lớp routing nhiều hơn mức máy có thể xử lý xong trong thời gian hợp lý. Ở cấu hình $N = 2$ (4 PE), luồng hoàn tất thành công.

Chẩn đoán ban đầu (dựa trên cấu trúc RTL, chưa xác nhận bằng log routing chi tiết): nhiều khả năng đây là hệ quả cộng hưởng của (a) số lượng cell tăng theo N^2 từ 4 PE ở $N = 2$ lên 64 PE ở $N = 8$, tức tăng 16 lần và (b) giao diện hiện tại đưa toàn bộ A và B vào top-level dưới dạng bus song song phẳng kích thước $N \times N$, khiến số lượng pin/wire ở top-level cũng tăng theo N^2 thay vì được dồn qua một giao diện scratchpad/streaming hẹp hơn. Đây là giả thuyết cần kiểm chứng lại khi có log congestion thực tế từ OpenLane, không phải kết luận cuối cùng.

6.3 Kết quả Chi tiết theo Từng Giai đoạn

Synthesis

Sau bước synthesis bằng Yosys, thiết kế top.v cấu hình $N = 2$ được ánh xạ sang thư viện standard cell sky130_fd_sc_hd của PDK SKY130. Kết quả synthesis cho thấy thiết kế có 2319 standard cells, với diện tích chip sau synthesis khoảng 23254.8032 μm^2 . Các thông số này phản ánh quy mô phân cứng của khối MAC Array INT8 sau khi RTL được chuyển sang netlist mức cổng.

Thông số	Giá trị
Số lượng cell sau synthesis	2319
Chip area sau synthesis	23254.8032 μm^2

Bảng 5. Kết quả Synthesis

```
===== YOSYS SYNTHESIS REPORT SEARCH =====
reports/synthesis/1-synthesis.AREA_0.stat.rpt:13:  Number of cells:                2319
reports/synthesis/1-synthesis.AREA_0.stat.rpt:70:  Chip area for module '\top': 23254.803200
```

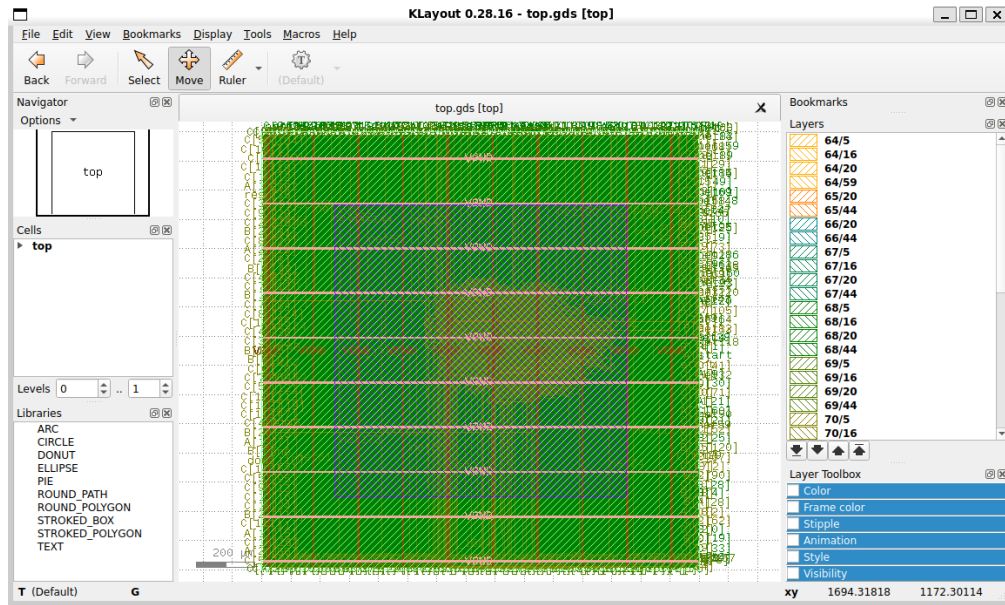
Hình 9. Kết quả Synthesis bằng Yosys

Placement và Routing

Sau synthesis, thiết kế được đưa qua các bước floorplanning, placement, CTS và routing trong OpenLane. Với cấu hình $N = 2$, thiết kế sử dụng die area 2.25 mm^2 , core area khoảng $2,198,543.58 \mu\text{m}^2$, floorplan utilization target 10% và placement target density 0.15. Việc chọn mật độ thấp giúp quá trình placement và routing ổn định hơn, đặc biệt với thiết kế có nhiều bus dữ liệu như MAC Array.

Thông số	Giá trị
Die area	2.25 mm^2
Core area	$2,198,543.58 \mu\text{m}^2$
FP_CORE_UTIL	10%
PL_TARGET_DENSITY	0.15
OpenDP utilization	1.08%
Cell/ mm^2 /Core Util	11306.67
CellPer_ mm^2	1130.67

Bảng 6. Kết quả Placement và Routing



Hình 10. Layout GDSII của MAC INT8 sau khi hoàn tất OpenLane fullflow.

Static Timing Analysis (STA)

Kết quả STA được thực hiện với chu kỳ clock mục tiêu 100 ns. Thiết kế cấu hình $N = 2$ có datapath nhỏ nên có khả năng đáp ứng timing ở tần số mục tiêu. Kết quả slack được dùng để đánh giá khả năng hoạt động đúng của mạch sau placement và routing.

Thông số	Giá trị
Flow status	flow completed
Clock period	100 ns
Fmax mục tiêu	10 MHz
WNS	0.0 ns
TNS	0.0 ns
SPEF WNS	0.0 ns
SPEF TNS	0.0 ns
Timing violation	Không có

Bảng 7. Kết quả Placement và Routing

Với WNS và TNS đều bằng 0.0 ns, thiết kế MAC Array INT8 cấu hình $N = 2$ đáp ứng ràng buộc thời gian ở clock period 100 ns. Điều này cho thấy datapath của cấu hình $N = 2$ đủ nhỏ để hoàn tất timing closure trong OpenLane.

Power

Công suất của thiết kế được OpenLane ước lượng sau quá trình triển khai vật lý dựa trên thư viện standard cell sky130_fd_sc_hd. Ở góc typical, công suất nội tại của cell là $0.00072 \mu\text{W}$, công suất chuyển mạch là $0.00137 \mu\text{W}$, và công suất rò là $1.19 \times 10^{-7} \mu\text{W}$. Tổng công suất ước lượng của thiết kế là khoảng $0.00209 \mu\text{W}$. Giá trị công suất này tương đối nhỏ do cấu hình triển khai trong báo cáo là $N = 2$, chỉ gồm 4 PE và số lượng cell sau synthesis là 2319 cells.

Thông số	Giá trị
Internal power	$0.00072 \mu\text{W}$
Switching power	$0.00137 \mu\text{W}$
Leakage power	$1.19 \times 10^{-7} \mu\text{W}$
Total power	$0.00209 \mu\text{W}$

Bảng 8. Kết quả Power

Kết quả power trong báo cáo được xem là giá trị ước lượng ở mức flow OpenLane, phụ thuộc vào giả định hoạt động mặc định của công cụ. Để đánh giá chính xác hơn, cần bổ sung file activity như VCD/SAIF từ mô phỏng thực tế.

6.4 Tổng kết PPA

Bảng 5 sẽ phản ánh cấu hình $N = 2$ cấu hình duy nhất đã route thành công tính đến thời điểm báo cáo.

Tham số	Giá trị
Cấu hình mảng (N)	2
Công nghệ / PDK	Sky 130nm
Diện tích core (μm^2)	2,198,543.58 μm^2
Số lượng cell	2319
Utilization (%)	10%
Fmax (MHz)	10 MHz
Tổng power (μW)	0.00209 μW
Số vi phạm DRC	0
LVS clean	yes

Bảng 9. Tổng kết PPA cấu hình $N = 2$

```
tritonRoute_violations: 0
Short_violations: 0
MetSpc_violations: 0
OffGrid_violations: 0
MinHole_violations: 0
Other_violations: 0
Magic_violations: 0
pin_antenna_violations: 88
net_antenna_violations: 80
lvs_total_errors: 0
cvc_total_errors: -1
klayout_violations: -1
GRT_REPAIR_ANTENNAS: 1
nhandeprai@DESKTOP-K1RQKRI:~/OpenLane/designs/npu_macarray/runs/N2_success$
```

Hình 11. kiểm tra DRC/LVS bằng các công cụ trong flow OpenLane

Chương 7: Kết luận và Hướng phát triển

7.1 Kết luận

Đồ án đã thiết kế và mô phỏng một mảng MAC INT8 weight-stationary, dự kiến làm datapath số “dự phòng”, chứng minh được trên silicon, cho một lõi NPU dựa trên CIM. Tóm lại, kết quả đạt được tính đến thời điểm báo cáo gồm: (1) RTL hoàn chỉnh cho PE, mảng MAC, và FSM điều khiển, tham số hóa cho $N = 8$; (2) kiểm tra chức năng bằng testbench tự kiểm tra với 5/5 test case PASS trong Verilator Verilog, xác nhận bằng cả log terminal và dạng sóng thực tế; (3) triển khai vật lý qua luồng RTL-to-GDSII mã nguồn mở OpenLane hoàn tất thành công ở cấu hình rút gọn $N = 2$, xác nhận toàn bộ luồng chạy được từ đầu đến cuối, trong khi cấu hình mục tiêu $N = 8$ hiện chưa route xong do số lớp routing cần thiết vượt khả năng máy hiện có nhiều khả năng liên quan đến số lượng cell tăng theo N^2 và giao diện bus song song phẳng hiện tại (Mục 6.2).

7.2 Hạn chế Tổng thể

- Khả năng route được ở quy mô $N = 8$ chưa được xác nhận; cần log congestion thực tế từ OpenLane để chẩn đoán chính xác nguyên nhân.
- Chế độ precision INT4 theo đặc tả A2 chưa được cài đặt; hiện tại mới chỉ hỗ trợ INT8.
- PE và chuỗi cộng dồn theo cột hoàn toàn tổ hợp, chưa pipeline hóa — có thể ảnh hưởng timing closure ở tần số cao.
- Giao diện bus hiện là bus song song phẳng $N \times N$, chưa phải giao diện memory-mapped/scratchpad thực tế như compiler CIM sẽ dùng.

- Bộ test hiện tại chỉ dùng ma trận giá trị đồng nhất, chưa kiểm tra dữ liệu keyword-spotting/ECG thật.
- Chưa thực hiện demo trên FPGA (board DE2 qua Quartus II).

7.3 Hướng Phát triển

- Giải quyết khả năng route ở $N = 8$: xem lại log congestion thực tế của OpenLane, sau đó cân nhắc (a) tách mảng $N \times N$ phẳng hiện tại thành các khối nhỏ hơn/pipeline theo tầng, và/hoặc (b) thay giao diện bus song song $N \times N$ bằng giao diện scratchpad/streaming hẹp hơn. Đây là ưu tiên trước mắt.
- Bổ sung hỗ trợ INT4 theo đúng đặc tả A2, cho phép chuyển đổi độ chính xác INT8/INT4.
- Pipeline hóa PE (thêm thanh ghi ở đầu ra) nếu STA cho thấy không đạt timing closure ở tần số mục tiêu.
- Thiết kế giao diện bus/scratchpad phù hợp (vd. AHB hoặc TileLink, như đã nhắc ở đề tài A1 liên quan) thay vì bus song song phẳng hiện tại.
- Mở rộng testbench với ma trận không đồng nhất ngẫu nhiên so với golden model phần mềm, và với dữ liệu feature keyword-spotting / ECG đã lượng tử hóa thật.
- Dựng thử thiết kế trên board DE2 qua Quartus II như một cách kiểm tra chức năng độc lập song song với luồng ASIC.

7.4 Mã nguồn

Mã nguồn RTL đầy đủ (pe.v, mac.v, top.v) và testbench (tb_top1.sv)

[Github](#)

Tài liệu Tham khảo

- [1] N. P. Jouppi et al., “In-Datacenter Performance Analysis of a Tensor Processing Unit,” Proc. 44th Annual International Symposium on Computer Architecture (ISCA), 2017, pp. 1–12.
- [2] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks,” IEEE Journal of Solid-State Circuits, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [3] NVIDIA, “NVDLA: NVIDIA Deep Learning Accelerator,” open-source project. [Online]. Available: <http://nvdla.org> and <https://github.com/nvdla/hw>.
- [4] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-based Tapeout-Proven Flow,” 2020 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), San Diego, CA, USA, 2020, pp. 1–6.